Freight Rate Prediction Assessment

1. Problem and data

Each load in this dataset is one truck shipment, and the task is to predict its posted price in dollars. Instead of predicting the price directly, my model predicts the rate per mile and then multiplies it by the trip distance.

I had two files to work with. `train_test.csv` has 48,000 priced loads from 1 January to 31 October 2025. `validation.csv` has 12,000 loads from 1 November to 31 December 2025, but no prices, so I can make predictions for it but can't check them myself.

The short source lines at the end of each part point to the saved evidence files listed in the appendix, where every number in this report is matched to the exact file and field it came from. The notebook prints the same files in the matching analysis sections.

Sources: D, V, K, Y

2. Data quality

Before modelling I checked both files for problems. I didn't delete or impute any rows. Missing values stay as NaN with a flag column next to them, so the model can still tell that something was missing.

Issue	train_test.csv	validation.csv	Treatment
Missing weight	300	165	Kept as NaN, flagged
Negative weight	292	145	Absolute value, flagged
Missing market index	374	249	Kept as NaN, flagged
Duplicate load IDs	0	0	Nothing removed
Quote behaviour	Changes by month	Flat	Not used in the final model
Cities not seen in training	-	8 cities, 1,447 loads (12.06%)	Model uses coordinates, not city names

The negative weights are worth a comment. Once I take the absolute value, they look almost exactly like the positive ones: both groups run from 5,000.00 to 47,500.00 lb, the means are 31,724.20 and 31,415.36 lb, and the medians are 31,821.50 and 31,493.50 lb. My reading is that these are sign errors, so I flipped them and kept a flag. I can't prove that, though. It is an inference from how similar the two groups are.

Sources: D, V, W, C, K

3. How I split the data

Since the real job is to predict a later period from earlier loads, I split by time rather than at random. The development set has 38,477 rows from 1 January to 31 August, and the test set has 9,523 rows from 1 September to 31 October.

For model selection I used monthly folds inside the development period. Each fold trains on all earlier months and is evaluated on the next one, from April through August. I started at April so

the first fold already has a quarter of history behind it; earlier folds would have had much less to learn from.

These five months don't all behave the same way when it comes to the quote signal (see Section 4). In order they were upside down, upside down, follows price, upside down and flat. August is the only flat one, which makes it the closest check for `validation.csv`.

I should be clear about how independent the test set really is. The current LightGBM model was scored on September and October only once, after I had locked every choice, and all feature and setting decisions came from the January to August folds. Still, those test months weren't completely untouched. An older checkpoint had already scored Ridge and log-target Ridge on them, and the original notebook I was given for this finalization explored them too, although it never scored a holdout model there.

Sources: G, F, B, V, PR, L, E

4. The quote signal

At first glance `quote_signal` looks like a useful feature. Looking closer, though, its relationship with distance keeps changing from month to month. To measure this I computed, for each day, the correlation between the quote and log distance, and grouped the months like this:

- below -0.5: *follows price* (short trips get a higher quote per mile, as they do in real prices)
- above 0.5: *upside down*
- anything in between: *flat*

The names only describe how the quote moves with distance. They say nothing about whether the quote is accurate.

Pattern	Months	Daily correlation	Short-trip quote	Long-trip quote
Follows price	Jan, Feb, Mar, Jun, Sep	-0.89 to -0.72	$2.64 to $2.93	$1.90 to $2.09
Upside down	Apr, May, Jul, Oct	0.70 to 0.88	$1.31 to $1.37	$2.10 to $2.14
Flat	Aug	-0.11 to 0.15	$2.04	$2.06

Short trips here are 200 miles or less and long trips are over 1,000 miles. The quote ranges are the lowest and highest monthly medians, not the spread of individual shipments.

The trouble is `validation.csv`. All 61 of its days are flat, with daily correlations between -0.17 and 0.21. In November and December the quote hardly changes with distance, so it can't reflect the higher per-mile cost of short trips that shows up in the actual prices. In `train_test.csv` the median rate per mile goes down as distance goes up, across the months. A model leaning on the quote would be relying on a pattern that isn't there in the period I need to predict, so I dropped `quote_signal` and everything derived from it.

August, the only labelled flat month, supports this. The model that used the quote had an MAE of $155.70 there, while the final model had $109.26.

Sources: MS, R, Q, X, training_rpm_medians.csv, N, F, S

5. Features and model

The final model has 13 inputs: `distance`, `weight`, `weight_missing`, `weight_was_negative`, `eq_DryVan`, `eq_Flatbed`, `eq_Reefer`, `pickup_lat`, `pickup_lon`, `delivery_lat`, `delivery_lon`, `market_index` and `market_index_missing`.

Distance gives the trip length, weight and the equipment columns describe the shipment, the four coordinates locate the pickup and delivery points, and the market index lets the predicted rate move with that supplied value. The flag columns record where something was missing or had to be corrected.

A few reasons behind the setup. Predicting rate per mile puts short and long trips on a similar scale, and keeping distance as an input still lets the model learn that short trips cost more per mile. I used Huber loss so that a handful of very large errors don't dominate training; the extreme prices are real observations, so I left them in the data. I went with LightGBM because its trees handle NaN values directly and don't need any scaling, which kept the pipeline simple.

I kept the tuning small on purpose. With only five validation months, a big search would mostly be fitting noise. I ran the check with seeds 42 and 43 and kept seed 42 for production. Setting alpha to 0.5 brought the two-seed mean MAE down from $108.16 to $104.94, an improvement of 2.98%. I also tried some extra features, but none of them passed the acceptance rule.

Sources: N, K, src/train.py, models/final_settings.json, M, T, E

6. Results

I report four metrics. MAE is the average absolute error in dollars and MAPE is the same idea as a percentage. RMSE puts more weight on large errors, and R^2 shows how much the model improves on simply predicting the mean. When I average over folds, each month counts equally.

Development folds

MAE ($)	Apr	May	Jun	Jul	Aug	Mean
Quote reference	403.31	403.40	82.85	415.77	317.26	324.52
Median RPM	222.23	224.23	257.30	241.65	260.52	241.19
LightGBM with quote	312.78	82.33	104.78	83.96	155.70	147.91
LightGBM core	125.43	157.34	210.86	122.53	95.45	142.32
LightGBM, default settings	98.36	91.98	142.76	97.56	109.93	108.12
LightGBM, tuned (final)	94.68	85.26	140.50	95.28	109.26	**104.99**

The tuned model has the lowest average error. It isn't the best in every month (the quote model does better in May, June and July, and the core model in August), but it has the lowest mean. The table uses the production seed, which gives a mean of $104.99; averaged over both tuning seeds it is $104.94. Mean MAPE is 4.57%. All of these are development validation results.

Sources: F, S, T

Test period (September and October)

Model	MAE ($)	MAPE	RMSE ($)	R^2
Selected LightGBM	160.21	6.78%	648.41	0.82
Quote reference	246.02	12.11%	711.80	0.78
Median RPM	256.95	11.63%	684.25	0.80

On the test months the selected model beats both baselines on every metric. Its error is clearly higher than in validation, though: an MAE of $160.21 and MAPE of 6.78%, compared with a mean validation MAE of $104.99. I haven't investigated the cause, and it is a fair warning that the real November and December error could also end up above the validation average. Neither September nor October has the flat quote pattern seen in `validation.csv`. Results by month, equipment type and distance group for every test model are printed in the notebook and saved with the overall scores.

Sources: H, S, B, V

Unseen cities

About 12% of the validation loads (1,447) involve 8 cities that never appear in training. To get a feel for how much that matters, I removed 8 randomly chosen cities from training and compared results on the same 1,226 August loads. The MAE barely moved, from $95.59 to $95.69. That is encouraging, but it's one random draw with less training data, so it doesn't guarantee the same for every set of unseen cities.

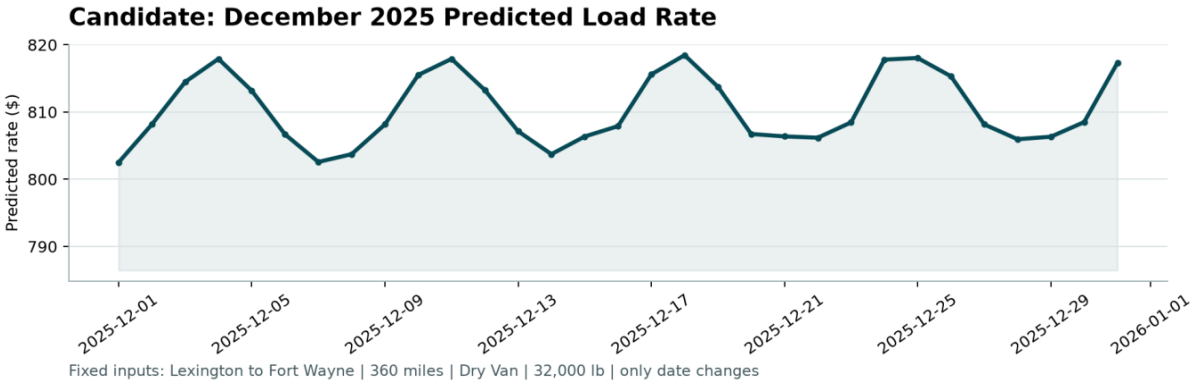
Sources: V, HC, U

Final predictions

Once the choices were settled I refitted the model on all of `train_test.csv`. The scorer accepted all 12,000 final predictions with none missing or nonpositive, so no prediction floor was needed.

Sources: J, P

7. December prediction chart



Fixed load: Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb. Only the date changes.

I produced this chart with the supplied scorer, unmodified, and it accepted all 31 fixed-load predictions. The inputs keep the original column order, fixed load values and date strings.

Of the model's inputs, only `market_index` changes from day to day. For each date I used the median index of the real loads in `validation.csv` on that day, and took the coordinates from the city lookup built from `train_test.csv`. This assumes the batch data is available. It is not a real-time forecast of the market index.

Weekday	Mean daily market index	Mean predicted price
Monday	0.85045	$805.01
Tuesday	0.92517	$808.23
Wednesday	1.00879	$816.13
Thursday	1.03243	$818.07
Friday	0.98573	$813.87
Saturday	0.90150	$807.17
Sunday	0.84303	$804.65

Predictions rise toward Thursday and fall back toward Sunday. That weekly pattern comes entirely from the daily median market index of the real December loads. I'm only describing it here, not claiming what drives the index.

Overall the line doesn't move much, staying between $802.49 and $818.45. As a rough reference, Dry Van loads of 300 to 420 miles in the labelled data had monthly median prices between $791.74 and $872.38, with an overall median of $833.97. That's context only, not a check on the December predictions.

Sources: J, Y, Z, P, WD, DY, MS

8. Limitations

There are a few things to keep in mind when reading these results.

The biggest is the gap between validation and test. September and October gave an MAE of $160.21 and MAPE of 6.78%, against $104.99 in validation. I didn't find out why, and the November and December error could be higher as well.

I also couldn’t check much for the target period itself. August is the only labelled month with flat quotes, the prices in `validation.csv` aren’t available, and I don’t know what final metric the assessors will use. The unseen-city check is based on a single random draw, and earlier exposure to the test months limits how independent that test is.

Some of the data handling rests on assumptions. Treating negative weights as sign errors is an inference, and I assume the market index is known at quote time. The December chart moves only because of the supplied daily index.

Finally, feature importance shows how the trees split the training data, not what actually causes prices. Distance has the largest share of gain, at 57.05%.

Sources: H, S, B, V, SC, U, L, W, Y, P, I

On the code side, the repository keeps the shared feature logic, training, evaluation and output generation in separate modules. The notebook imports that logic, and the saved evidence records the settings, the split, test exposure and scorer output.

Sources: src/features.py, src/train.py, src/evaluate.py, src/run.py

Appendix A: Evidence files

Each key points to a file path relative to the repository root.

Key	File	Key	File
D	reports/evidence/training_summary.json	**L**	reports/evidence/holdout_ledger.json
V	reports/evidence/external_summary.json	**M**	models/selection.json
W	reports/evidence/weight_comparison.csv	**J**	reports/evidence/scorer_log.txt
Q	reports/evidence/training_quote_medians.csv	**G**	reports/evidence/split.csv
B	reports/evidence/training_behaviour.csv	**N**	models/features.json
R	reports/evidence/training_correlation_ranges.csv	**Z**	data/december_chart_inputs.original.csv
X	reports/evidence/external_correlation_ranges.csv	**HC**	reports/evidence/hidden_cities.json
C	reports/evidence/cleaning_summary.json	**O**	reports/evidence/comparable_monthly_medians.csv
F	reports/evidence/cv_folds.csv	**K**	src/features.py
A	reports/evidence/cv_summary.csv	**Y**	src/run.py
S	reports/evidence/selected_cv_folds.csv	**E**	src/evaluate.py
T	reports/evidence/tuning.csv	**SC**	score.py
H	reports/evidence/holdout_metrics.csv	**MS**	reports/evidence/method_settings.json
U	reports/evidence/unseen_city_test.csv	**PR**	reports/evidence/prior_holdout_exposure.json
P	reports/evidence/prediction_summary.json	**WD**	reports/evidence/december_weekday_pattern.csv
I	reports/evidence/feature_importance.csv	**DY**	reports/evidence/december_daily_pattern.csv

Appendix B: Where each number comes from

Each number in the report, with the file and field it was read from.

Claim	Value	File / field
Labeled rows	48,000	training_summary.json / rows
Labeled start	2025-01-01	training_summary.json / start
Labeled end	2025-10-31	training_summary.json / end
External rows	12,000	external_summary.json / rows
External start	2025-11-01	external_summary.json / start
External end	2025-12-31	external_summary.json / end
Labeled missing weight	300	training_summary.json / missing_weight
External missing weight	165	external_summary.json / missing_weight
Labeled negative weight	292	training_summary.json / negative_weight
External negative weight	145	external_summary.json / negative_weight
Labeled missing market index	374	training_summary.json / missing_market_index
External missing market index	249	external_summary.json / missing_market_index
Labeled duplicate IDs	0	training_summary.json / duplicate_ids
External duplicate IDs	0	external_summary.json / duplicate_ids

Claim	Value	File / field
Unseen city count	8	external_summary.json / unseen_city_count
Affected loads	1,447	external_summary.json / affected_loads
Affected share	12.06%	external_summary.json / affected_pct
Both weight ranges	5,000.00 to 47,500.00 lb	weight_comparison.csv / min and max
Weight means	31,724.20 and 31,415.36 lb	weight_comparison.csv / mean
Weight medians	31,821.50 and 31,493.50 lb	weight_comparison.csv / 50%
Development rows	38,477	split.csv / Development
Development start / end	2025-01-01 / 2025-08-31	split.csv / Development
Test rows	9,523	split.csv / Test
Test start / end	2025-09-01 / 2025-10-31	split.csv / Test
Follows-price threshold	-0.5	method_settings.json / quote_correlation_lower
Upside-down threshold	0.5	method_settings.json / quote_correlation_upper
Follows price: correlation range	-0.89 to -0.72	training_correlation_ranges.csv / follows price
Follows price: short-trip quote	$2.64 to $2.93	training_quote_medians.csv / <=200
Follows price: long-trip quote	$1.90 to $2.09	training_quote_medians.csv / >1000
Upside down: correlation range	0.70 to 0.88	training_correlation_ranges.csv / upside down
Upside down: short-trip quote	$1.31 to $1.37	training_quote_medians.csv / <=200
Upside down: long-trip quote	$2.10 to $2.14	training_quote_medians.csv / >1000
Flat: correlation range	-0.11 to 0.15	training_correlation_ranges.csv / flat
Flat: short-trip quote	$2.04	training_quote_medians.csv / <=200
Flat: long-trip quote	$2.06	training_quote_medians.csv / >1000
Short-trip boundary	200 miles	training_quote_medians.csv / <=200
Long-trip boundary	1,000 miles	training_quote_medians.csv / >1000
External flat days	61	external_summary.json / flat_days
External correlation range	-0.17 to 0.21	external_correlation_ranges.csv / flat
Feature count	13	models/features.json / complete list
Tuning seeds	42 and 43	models/selection.json / seeds
Production seed	42	models/selection.json / params.random_state
Selected alpha	0.5	models/selection.json / params.alpha
Default two-seed MAE	$108.16	tuning.csv / default.mean_MAE
Chosen two-seed MAE	$104.94	tuning.csv / alpha_0.5.mean_MAE
Tuning improvement	2.98%	tuning.csv / alpha_0.5.improvement_pct
Quote reference, Apr–Aug MAE	403.31, 403.40, 82.85, 415.77, 317.26	cv_folds.csv / Quote reference / 2025-04 … 2025-08
Median RPM, Apr–Aug MAE	222.23, 224.23, 257.30, 241.65, 260.52	cv_folds.csv / Median RPM / 2025-04 … 2025-08
LightGBM with quote, Apr–Aug MAE	312.78, 82.33, 104.78, 83.96, 155.70	cv_folds.csv / LightGBM with quote / 2025-04 … 2025-08
LightGBM core, Apr–Aug MAE	125.43, 157.34, 210.86, 122.53, 95.45	cv_folds.csv / LightGBM core / 2025-04 … 2025-08

Claim	Value	File / field
LightGBM default, Apr–Aug MAE	98.36, 91.98, 142.76, 97.56, 109.93	`cv_folds.csv / LightGBM final / 2025-04 … 2025-08`
Selected, Apr–Aug MAE	94.68, 85.26, 140.50, 95.28, 109.26	`selected_cv_folds.csv / 2025-04 … 2025-08`
Mean MAE: quote ref. / median RPM	324.52 / 241.19	`cv_folds.csv / mean by model`
Mean MAE: with quote / core / default	147.91 / 142.32 / 108.12	`cv_folds.csv / mean by model; cv_summary.csv`
Selected mean MAE	$104.99	`selected_cv_folds.csv / mean MAE`
Selected mean MAPE	4.57%	`selected_cv_folds.csv / mean MAPE`
Selected LightGBM test MAE / MAPE	160.21 / 6.78%	`holdout_metrics.csv / overall / Selected LightGBM`
Selected LightGBM test RMSE / R^2	648.41 / 0.82	`holdout_metrics.csv / overall / Selected LightGBM`
Quote reference test MAE / MAPE	246.02 / 12.11%	`holdout_metrics.csv / overall / Quote reference`
Quote reference test RMSE / R^2	711.80 / 0.78	`holdout_metrics.csv / overall / Quote reference`
Median RPM test MAE / MAPE	256.95 / 11.63%	`holdout_metrics.csv / overall / Median RPM`
Median RPM test RMSE / R^2	684.25 / 0.80	`holdout_metrics.csv / overall / Median RPM`
Hidden-city count	8	`hidden_cities.json`
Hidden-city evaluation rows	1,226	`unseen_city_test.csv / both conditions`
Seen-city MAE	$95.59	`unseen_city_test.csv / Cities seen in training`
Hidden-city MAE	$95.69	`unseen_city_test.csv / Cities hidden from training`
Accepted final rows	12,000	`scorer_log.txt / first line`
Accepted December rows	31	`scorer_log.txt / second line`
December minimum / maximum	$802.49 / $818.45	`prediction_summary.json / december_min, december_max`
Comparable distance bounds	300 and 420 miles	`method_settings.json / comparable_distance_bounds`
Comparable monthly min / max	$791.74 / $872.38	`prediction_summary.json / comparable_monthly_median_min, _max`
Comparable overall median	$833.97	`prediction_summary.json / comparable_median`
Distance gain share	57.05%	`feature_importance.csv / distance.gain_pct`
Mon: index / prediction	0.85045 / $805.01	`december_weekday_pattern.csv / Monday`
Tue: index / prediction	0.92517 / $808.23	`december_weekday_pattern.csv / Tuesday`
Wed: index / prediction	1.00879 / $816.13	`december_weekday_pattern.csv / Wednesday`
Thu: index / prediction	1.03243 / $818.07	`december_weekday_pattern.csv / Thursday`
Fri: index / prediction	0.98573 / $813.87	`december_weekday_pattern.csv / Friday`
Sat: index / prediction	0.90150 / $807.17	`december_weekday_pattern.csv / Saturday`
Sun: index / prediction	0.84303 / $804.65	`december_weekday_pattern.csv / Sunday`

File names in Appendix B are under `reports/evidence/` *unless a full path is shown.*

Methods and interpretations

Statement	Evidence
No deletion or imputation; absolute weights with flags	`src/features.py / clean_data; reports/evidence/cleaning_summary.json`
No quote, calendar or price-history features in the final model	`models/features.json; src/features.py / build_features`
Rate-per-mile target; Huber LightGBM; no scaling	`src/train.py / fit_model; models/final_settings.json`
Development-only selection; guarded final test; prior exposure	`src/evaluate.py; reports/evidence/holdout_ledger.json`
Final template merge preserves ID order	`src/run.py / run_final; validation_predictions.csv`
December backup, coordinate lookup, daily median index, unchanged fixed columns	`src/run.py / run_final; data/december_chart_inputs.original.csv; reports/evidence/december_augmented.csv`
Only market index varies in December	`reports/evidence/prediction_summary.json / varying_december_features`
External labels and final metric unavailable	`data/validation.csv column header; score.py / main`
Rate per mile falls with distance in observed group medians	`reports/evidence/training_rpm_medians.csv; reports/evidence/rpm_sanity.csv`
External unseen-city list and affected rows	`reports/evidence/external_summary.json / unseen_cities, affected_loads`
Feature importance is training gain, not causal evidence	`src/run.py / feature_importance call`